

Guía de Laboratorio 2

Usos de Queues y SW Timers en FreeRTOS

alaprovitta@unc.edu.ar

Marzo 2026

Objetivo General

Comprender la comunicación de datos y el determinismo, incorporando recursos que los preparen para sistemas de tiempo real más complejos.

Recursos Necesarios

- Hardware: Microcontrolador STM32, 4 LEDs (LED1 a LED4) y 1 Pulsador.
- Software: STM32CubeIDE.
- Librerías permitidas: `FreeRTOS.h`, `queue.h` y `timer.h`.

Desafíos Prácticos

Desafío 1: Comunicación por Queues

Objetivo: Transferir datos entre tareas de forma segura.

Recursos: `queue.h`.

Comportamiento: Una tarea "Productora" lee el estado del pulsador y mantiene un contador de cuántas veces fue presionado. Con cada "click" envía este valor (un `uint16_t`) a través de una cola. Una tarea "Consumidora" recibe el valor y lo muestra en los 4 LEDs representando el número en binario (o su complemento de 16).

Análisis: ¿Qué sucede si la tarea consumidora es más lenta que la productora? ¿Cómo afecta el tamaño de la cola?

Desafío 2: Paso de Estructuras

Objetivo: Gestionar múltiples datos en un solo mensaje.

Recursos: `queue.h`.

Comportamiento: Crear una estructura que contenga un ID de LED y un comando (ENCENDER/APAGAR). La tarea principal envía estas estructuras a una cola. Una tarea de gestión de periféricos debe procesar la cola y ejecutar la acción en el LED correspondiente. Generar múltiples patrones de encendido en los 4 leds desde la tarea principal.

Análisis: Comparar el uso de memoria de enviar una estructura por copia frente a enviar un puntero a la estructura.

Desafío 3: Multi-recepción con Queue Sets

Objetivo: Gestionar múltiples fuentes de datos asincrónicas desde una única tarea receptora sin pérdida de eventos.

Servicios requeridos: `xQueueCreateSet()`, `xQueueAddToSet()`, `xQueueSelectFromSet()`.

Comportamiento: Dos tareas distintas deben enviar información a una tarea "Procesadora". Se deben crear dos colas distintas: `Cola_Velocidad` (envía valores de tiempo en ms) y `Cola_Modo` (envía comandos como 'D' para derecha o 'I' para izquierda).

- La tarea "Procesadora" debe permanecer bloqueada a la esperar datos de ambas colas simultáneamente.
- Los 4 LEDs deben realizar una secuencia (ej: Knight Rider). Si llega un dato por `Cola_Velocidad`, se actualiza la frecuencia de la secuencia. Si llega por `Cola_Modo`, se cambia el sentido de giro.
- La tarea que envía el Modo, debe cambiar el sentido de giro con cada "click" del pulsador
- La tarea que envía la Velocidad debe enviar un nuevo valor aleatorio (entre 100 y 300ms) cada intervalos de 5 segundos.

Análisis: ¿Qué ventaja tiene usar un Queue Set frente a realizar un "Polling" con un `xQueueReceive` de tiempo de espera cero sobre cada cola? ¿Cómo influye el tamaño de las colas individuales en el comportamiento del Set?

Desafío 4: Distribución de Datos con Queue Peek

Objetivo: Permitir que múltiples tareas accedan a la misma información sin "consumirla" de la cola, permitiendo un procesamiento paralelo de un mismo evento.

Servicios requeridos: `xQueuePeek()`, `xQueueOverwrite()`.

Comportamiento:

- Crear una única cola `Cola_Global` con capacidad para 1 solo elemento (tipo `uint16_t`).
- **Tarea Productora:** Lee el pulsador. Cada vez que se presiona, incrementa un contador y lo pone en la cola (si la cola está llena, debe sobrescribir el valor anterior).
- **Tareas Consumidoras (A y B):** Ambas tareas deben estar pendientes de la misma cola.
 - La **Tarea A** debe usar `xQueuePeek()` para leer el valor. Si el valor es par, hace parpadear el LED1.
 - La **Tarea B** debe usar `xQueuePeek()` para leer el valor. Si el valor es impar, hace parpadear el LED2.
- Solo una tercera tarea de "Limpieza" (o una de las anteriores bajo cierta condición) debe finalmente usar `xQueueReceive()` para vaciar la cola después de que ambas hayan procesado el dato.

Análisis: ¿Qué sucede si una tarea usa `xQueueReceive()` en lugar de `xQueuePeek()`? ¿Cómo se aseguran de que ambas tareas leyeron el mismo dato antes de que este sea eliminado de la cola?

Desafío 5: Temporizadores de Seguridad (One-Shot)

Objetivo: Implementar un mecanismo de "Watchdog" de software para desactivar procesos tras inactividad.

Servicios requeridos: `xTimerCreate()` con `uxAutoReload = pdFALSE`, `xTimerStart()`, `xTimerReset()`.

Comportamiento: El sistema inicia con el LED1 parpadeando (vía tarea).

- Al presionar el Pulsador (por interrupción), se enciende el LED2 y se dispara un Software Timer de **un solo disparo (One-Shot)** configurado a 5 segundos.
- Si el pulsador se presiona nuevamente antes de que expiren los 5 segundos, el Timer debe reiniciarse (`xTimerReset()`).
- Al expirar el tiempo, la función de *callback* del Timer debe apagar el LED2 y encender el LED3 durante 1 segundo como señal de "Timeout".

Análisis: ¿Qué sucede con el LED2 si el usuario presiona el botón repetidamente cada 2 segundos? ¿LED1 se vio afectado por la lógica de LED2?... ¿La función de callback puede utilizar `vTaskDelay()`? ¿Por qué?

Desafío 6: Metrónomo de Alerta (Auto-Reload)

Objetivo: Gestionar eventos cíclicos sin crear tareas adicionales, optimizando el uso de la RAM.

Servicios requeridos: `xTimerCreate()` con `uxAutoReload = pdTRUE`, `xTimerChangePeriod()`.

Comportamiento:

- Crear un Software Timer **Auto-Reload** con un período inicial de 1000ms.
- En cada expiración (callback), el Timer debe conmutar (toggle) el LED4.
- El Pulsador debe funcionar como un selector de frecuencia: cada pulsación debe reducir el período del Timer a la mitad (1000ms -> 500ms -> 250ms -> 125ms) y, al llegar al mínimo, volver a 1000ms.

Análisis: Comparar el consumo de Stack de este diseño frente a una tarea que haga lo mismo con un bucle y `vTaskDelay()`. ¿Dónde reside la lógica de conmutación del LED en este desafío?

Desafío 7: Sistema de Alarmas Escalonadas (Combinado)

Objetivo: Orquestar múltiples temporizadores para crear secuencias complejas de control.

Servicios requeridos: Uso conjunto de `xTimerStop()`, `xTimerStart()` y manipulación de estados dentro de los callbacks.

Comportamiento:

- El sistema debe simular un proceso de "Armado de Alarma":
 1. Al presionar el Pulsador, se inicia un Timer **Auto-Reload** que hace parpadear el LED1 rápidamente (indicador de pre-armado).
 2. Simultáneamente, se inicia un Timer **One-Shot** de 10 segundos.
 3. Cuando el Timer One-Shot expira (proceso completado), este debe **detener** al Timer Auto-Reload del LED1 y encender de forma fija el LED2 (indicador de sistema armado).
- Si se presiona el pulsador durante los 10 segundos de espera, ambos timers deben cancelarse y el sistema vuelve al estado inicial (LEDs OFF).

Análisis: ¿Cómo se comunican los timers entre sí o con el resto del sistema? ¿Es seguro modificar el periodo o detener un timer desde el callback de otro timer?